Ejercicio Parctico Clase 2 Grupo 4 Integrantes:

- Danny Sagal
- Lenin Miño
- Jhonny Ramirez

En esta actividad, llevaremos a cabo una serie de pruebas para evaluar la capacidad de nuestro modelo de clasificación de imágenes sobre una base de datos que debe ser creada por cada estudiante/grupo. El objetivo es identificar correctamente si una imagen contiene el rostro de un estudiante específico o si pertenece a un fondo. Para ello, hemos definido dos etiquetas principales:

- Alumno (Nombre del estudiante)
- Fondo

Empezamos descargando las librerías necesarias para que corra nuestro código

In [ ]:
```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from keras.applications import VGG16
from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.utils import to_categorical
from sklearn.model_selection import train_test_split
from tensorflow.keras.layers import Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

```
2025-11-08 09:56:17.060574: I tensorflow/core/platform/cpu_feature_guard.cc:210] Thi
s TensorFlow binary is optimized to use available CPU instructions in performance-cr
itical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorF
low with the appropriate compiler flags.
```

Para iniciar nuestro ejercicio vamos a usar la webcam del compañero seleccionado: Danny Sagal para capturar una secuencia de fotos de su rostro y almacenarlas

In [ ]:
```
# 1. Corregimos el nombre del paquete para la instalación
%pip install opencv-python

import cv2
import os
import time

def capture_images(duration, output_folder):
    # Crear carpeta si no existe
    if not os.path.exists(output_folder):
        os.makedirs(output_folder)
```

```
    # Abrir la webcam
    cap = cv2.VideoCapture(0)

    # Verificamos si la cámara realmente se abrió
    if not cap.isOpened():
        print("Error: No se pudo acceder a la webcam.")
        return

    start_time = time.time()
    count = 0  # Usamos un contador para evitar colisión de nombres

    print(f"Capturando imágenes por {duration} segundos...")

    try:
        while (time.time() - start_time) < duration:
            ret, frame = cap.read()
            if not ret:
                print("Error: Fallo al capturar el frame.")
                break

            # Guardar con timestamp Y contador para que cada nombre sea único
            image_name = f'img_{int(time.time())}_{count}.jpg'
            image_path = os.path.join(output_folder, image_name)

            cv2.imwrite(image_path, frame)
            count += 1

            # Pequeña pausa para no saturar el procesador
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

    finally:
        # Esto asegura que la cámara se libere incluso si hay un error
        cap.release()
        cv2.destroyAllWindows()
        print(f"Proceso finalizado. Se capturaron {count} imágenes en '{output_fold

# Ejecución
capture_images(duration=10, output_folder='imagenes_capturadas')
```

El siguiente código me permitirá descargar 50 rostros aleatorios de personas de diferentes etnias para educar el modelo

In [ ]:

```
import os
import requests
import time

def descargar_rostros(cantidad=50, ruta_destino=''):
    # 1. Crear el directorio si no existe
    if not os.path.exists(ruta_destino):
        os.makedirs(ruta_destino)
        print(f"✅ Carpeta creada en: {ruta_destino}")

    # URL de un servicio que genera rostros aleatorios
    # Usamos un parámetro de tiempo para evitar que el caché nos dé siempre la mism
```

```python
    url_base = "https://i.pravatar.cc/300"

    print(f"🚀 Iniciando descarga de {cantidad} imágenes...")

    for i in range(1, cantidad + 1):
        try:
            # Añadimos un parámetro aleatorio al final de la URL para forzar una im
            response = requests.get(f"{url_base}?u={i}")

            if response.status_code == 200:
                nombre_archivo = f"rostro_{i:03d}.jpg"
                path_completo = os.path.join(ruta_destino, nombre_archivo)

                with open(path_completo, 'wb') as f:
                    f.write(response.content)

                print(f"[{i}/{cantidad}] Descargada: {nombre_archivo}")

            # Pausa de medio segundo para no saturar el servidor y ser "buenos ciud
            time.sleep(0.5)

        except Exception as e:
            print(f" Error en la descarga {i}: {e}")

# --- CONFIGURACIÓN ---
directorio_fondo = r'C:\MAESTRIA\VISION_COMPUTADORA_C1\CLASEII\imagenes_capturadas\

# Ejecutar la descarga
descargar_rostros(50, directorio_fondo)

print("\n🎉 ¡Proceso terminado! Ya tienes tus 50 rostros listos.")
```

```
🚀 Iniciando descarga de 50 imágenes...
[1/50] Descargada: rostro_001.jpg
[2/50] Descargada: rostro_002.jpg
[3/50] Descargada: rostro_003.jpg
[4/50] Descargada: rostro_004.jpg
[5/50] Descargada: rostro_005.jpg
[6/50] Descargada: rostro_006.jpg
[7/50] Descargada: rostro_007.jpg
[8/50] Descargada: rostro_008.jpg
[9/50] Descargada: rostro_009.jpg
[10/50] Descargada: rostro_010.jpg
[11/50] Descargada: rostro_011.jpg
[12/50] Descargada: rostro_012.jpg
[13/50] Descargada: rostro_013.jpg
[14/50] Descargada: rostro_014.jpg
[15/50] Descargada: rostro_015.jpg
[16/50] Descargada: rostro_016.jpg
[17/50] Descargada: rostro_017.jpg
[18/50] Descargada: rostro_018.jpg
[19/50] Descargada: rostro_019.jpg
[20/50] Descargada: rostro_020.jpg
[21/50] Descargada: rostro_021.jpg
[22/50] Descargada: rostro_022.jpg
[23/50] Descargada: rostro_023.jpg
[24/50] Descargada: rostro_024.jpg
[25/50] Descargada: rostro_025.jpg
[26/50] Descargada: rostro_026.jpg
[27/50] Descargada: rostro_027.jpg
[28/50] Descargada: rostro_028.jpg
[29/50] Descargada: rostro_029.jpg
[30/50] Descargada: rostro_030.jpg
[31/50] Descargada: rostro_031.jpg
[32/50] Descargada: rostro_032.jpg
[33/50] Descargada: rostro_033.jpg
[34/50] Descargada: rostro_034.jpg
[35/50] Descargada: rostro_035.jpg
[36/50] Descargada: rostro_036.jpg
[37/50] Descargada: rostro_037.jpg
[38/50] Descargada: rostro_038.jpg
[39/50] Descargada: rostro_039.jpg
[40/50] Descargada: rostro_040.jpg
[41/50] Descargada: rostro_041.jpg
[42/50] Descargada: rostro_042.jpg
[43/50] Descargada: rostro_043.jpg
[44/50] Descargada: rostro_044.jpg
[45/50] Descargada: rostro_045.jpg
[46/50] Descargada: rostro_046.jpg
[47/50] Descargada: rostro_047.jpg
[48/50] Descargada: rostro_048.jpg
[49/50] Descargada: rostro_049.jpg
[50/50] Descargada: rostro_050.jpg

🎉 ¡Proceso terminado! Ya tienes tus 50 rostros listos.
```

Cargamos las imágenes del alumno Danny Sagal y el fondo, en fondo colocamos tambien los 50 rostros descargados en el punto anterior.

In [48]:
```
import cv2
import os
import shutil
import random

# 1. RUTAS DE ORIGEN (Kaggle) Y DESTINO (Tu carpeta de trabajo)
ruta_fuente = r'C:\MAESTRIA\VISION_COMPUTADORA_C1\CLASEII\imagenes_capturadas'
ruta_destino_base = r'C:\MAESTRIA\VISION_COMPUTADORA_C1\CLASEII\DB2'

# Definimos las clases y cantidad
categorias = {'Alumno': 'Al_reduced', 'Fondo': 'Fondo_reduced'}
cantidad_fotos = 80

def crear_dataset_reducido():
    # Creamos la carpeta base DB si no existe
    if not os.path.exists(ruta_destino_base):
        os.makedirs(ruta_destino_base)

    for origen_nombre, destino_nombre in categorias.items():
        # Construimos rutas completas
        path_origen = os.path.join(ruta_fuente, origen_nombre)
        path_destino = os.path.join(ruta_destino_base, destino_nombre)

        # 2. CREAR CARPETA DE DESTINO
        if os.path.exists(path_destino):
            # Si ya existe, la borramos para empezar de cero (opcional)
            shutil.rmtree(path_destino)
        os.makedirs(path_destino)

        # 3. FILTRAR Y SELECCIONAR FOTOS
        # Listamos archivos evitando archivos de sistema (como Thumbs.db)
        archivos = [f for f in os.listdir(path_origen)
                    if f.lower().endswith(('.jpg', '.jpeg', '.png'))]

        # Selección aleatoria
        seleccion = random.sample(archivos, min(len(archivos), cantidad_fotos))

        print(f"Copiando {len(seleccion)} fotos de {origen_nombre} a {destino_nombr

        # 4. COPIAR FÍSICAMENTE
        for nombre in seleccion:
            shutil.copy(os.path.join(path_origen, nombre),
                        os.path.join(path_destino, nombre))

    print("\n" + "="*40)
    print("¡PROCESO FINALIZADO CON ÉXITO!")
    print(f"Tus datos están listos en: {ruta_destino_base}")
    print("="*40)

# Ejecutar la función
crear_dataset_reducido()
```

```
Copiando 80 fotos de Alumno a Al_reduced...
Copiando 58 fotos de Fondo a Fondo_reduced...

========================================
¡PROCESO FINALIZADO CON ÉXITO!
Tus datos están listos en: C:\MAESTRIA\VISION_COMPUTADORA_C1\CLASEII\DB2
========================================
```

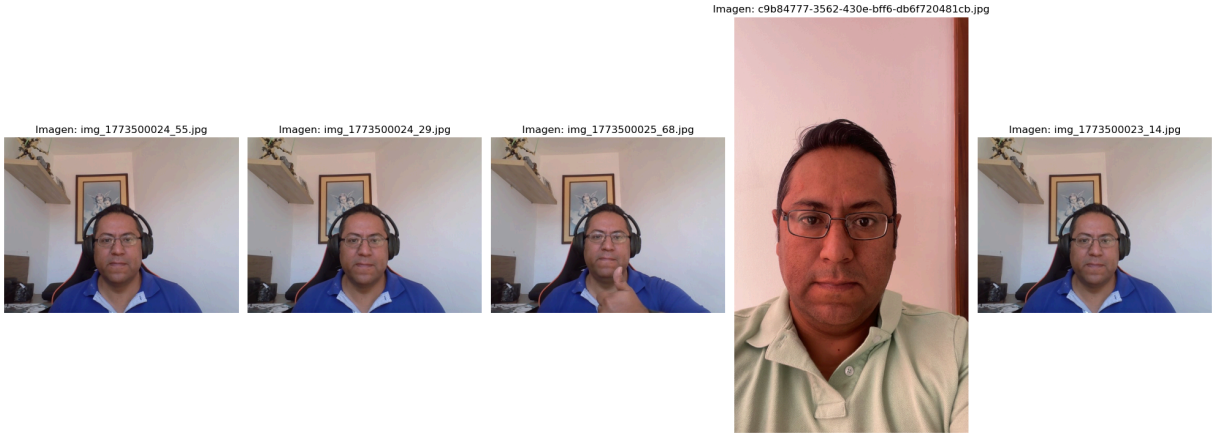
El el siguiente codigo generamos las etiquetas del modelo:

Alumno para la foto del estudiante seleccionado Fondo: imagenes de rostros diferentes aleatorios para entrenar el modelo

1.Foto del estudiante que no se encuentre en la carpeta de entrenamiento (tómate una foto el día de la presentación usando tu webcam). aqui tambien agregamos varias fotos del estudiante con diferente ropa y accesorios para mejorar el entrenamiento de la red.

In [80]:
```python
import cv2
import os
import random
import matplotlib.pyplot as plt

def visualizar_imagenes_azar(ruta_carpeta, cantidad=5):
    # 1. Obtener lista de archivos en la carpeta
    archivos = [f for f in os.listdir(ruta_carpeta) if f.lower().endswith(('.png',

    if len(archivos) < cantidad:
        print(f"La carpeta solo contiene {len(archivos)} imágenes.")
        cantidad = len(archivos)

    # 2. Seleccionar imágenes al azar
    imagenes_seleccionadas = random.sample(archivos, cantidad)

    # 3. Configurar la visualización con Matplotlib
    plt.figure(figsize=(20, 10))

    for i, nombre_archivo in enumerate(imagenes_seleccionadas):
        ruta_completa = os.path.join(ruta_carpeta, nombre_archivo)
        img = cv2.imread(ruta_completa)

        if img is not None:
            # Convertir de BGR (OpenCV) a RGB (Matplotlib)
            img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

            # Crear sub-gráficos en una sola fila
            plt.subplot(1, cantidad, i + 1)
            plt.imshow(img_rgb)
            plt.title(f"Imagen: {nombre_archivo}")
            plt.axis('off') # Ocultar los ejes (coordenadas)
        else:
            print(f"No se pudo cargar: {nombre_archivo}")

    plt.tight_layout()
```

```
    plt.show()

# Ejecutar la función con tu ruta específica
ruta_alumno = r'C:\MAESTRIA\VISION_COMPUTADORA_C1\CLASEII\imagenes_capturadas\Alumn
visualizar_imagenes_azar(ruta_alumno, cantidad=5)
```



In [49]:

```
import cv2
import os
import numpy as np

def load_images(folder):
    # Listas locales para esta carpeta
    images = []
    labels = []

    # Extraemos el nombre de la subcarpeta (Al_reduced o Fondo_reduced)
    folder_name = os.path.basename(folder)

    if not os.path.exists(folder):
        print(f"Error: No se encuentra la ruta {folder}")
        return images, labels

    for filename in os.listdir(folder):
        img_path = os.path.join(folder, filename)
        img = cv2.imread(img_path)

        if img is not None:
            # Redimensionamos al estándar de VGG-16
            img = cv2.resize(img, (224, 224))
            images.append(img)

            # --- NUEVA LÓGICA DE ETIQUETADO ---
            if folder_name == 'Al_reduced':
                labels.append(1) # 1 representa la presencia de un ALUMNO
            elif folder_name == 'Fondo_reduced':
                labels.append(0) # 0 representa solo el FONDO

    return images, labels

# 1. CARGAMOS LAS IMÁGENES DE ALUMNOS
print("Procesando carpeta de Alumnos...")
alumnos_images, alumnos_labels = load_images('C:/MAESTRIA/VISION_COMPUTADORA_C1/CLA
```

```python
# 2. CARGAMOS LAS IMÁGENES DE FONDOS
print("Procesando carpeta de Fondos...")
fondos_images, fondos_labels = load_images('C:/MAESTRIA/VISION_COMPUTADORA_C1/CLASE

# 3. COMBINAMOS TODO EN LOS TENSORES FINALES
# Juntamos las listas de fotos y las de etiquetas
images = np.array(alumnos_images + fondos_images)
labels = np.array(alumnos_labels + fondos_labels)

print("-" * 30)
print(f"✅ Carga finalizada.")
print(f"Total Alumnos: {len(alumnos_images)}")
print(f"Total Fondos: {len(fondos_images)}")
print(f"Forma del tensor 'images': {images.shape}")
```

```
Procesando carpeta de Alumnos...
Procesando carpeta de Fondos...
------------------------------
✅ Carga finalizada.
Total Alumnos: 80
Total Fondos: 58
Forma del tensor 'images': (138, 224, 224, 3)
```

In [50]:

```python
from keras.utils import to_categorical

# One-hot encoding de las etiquetas
labels = to_categorical(labels) # Convertimos las etiquetas a formato one-hot encod
labels
```

Out[50]:
```
array([[0., 1.],
       [0., 1.],
       [0., 1.],
       [
```

```
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [1., 0.],
       [1., 0.],
       [1., 0
```

```
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.]])
```

Dividmos los datos en conjuntos de entrenamiento y prueba:

In [51]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(images, labels, test_size=0.2)
```

Construimos y entrenamos la red neuronal convolucional VGG-16:

Un punto importante aquí es que para optimizar el tiempo de procesamiento de nuestra red, hemos generado un nuevo ambiente en Anaconda y activado nuestra tarjeta gráfica para que el procesamiento se distribuya tanto en el CPU como en la tarjeta y poder optimizar el tiempo de entrenamiento de la red.

In [52]:
```
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.callbacks import EarlyStopping
# Definimos el callback de EarlyStopping
early_stopping = EarlyStopping(monitor='val_loss', patience=10)

# Cargamos la red VGG16, asegurándonos de que se dejan fuera las capas superiores d
baseModel = VGG16(weights="imagenet", include_top=False, input_shape=(224, 224, 3))

# Construimos la capa que se colocará en la parte superior del modelo base (baseMod
model = Sequential()
model.add(baseModel)
model.add(Flatten())
model.add(Dense(256, activation='relu'))
```

```
model.add(Dropout(0.5))  # Agregamos una capa de Dropout con una tasa de 0.5
model.add(Dense(2, activation='softmax'))  # Dos clases (perro y gato), por lo que

# Congelamos las capas del modelo base para que no se actualicen durante el entrena
for layer in baseModel.layers:
    layer.trainable = False

# Compilamos el modelo
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy

# Entrenamos el modelo
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=300, batch_siz

# Evaluamos el modelo
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Loss: {loss}')
print(f'Accuracy: {accuracy}')
```

```
Epoch 1/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 14s 3s/step - accuracy: 0.7413 - loss: 8.6483 - val_accurac
y: 0.9643 - val_loss: 1.0984
Epoch 2/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 12s 3s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_acc
uracy: 1.0000 - val_loss: 0.0000e+00
Epoch 3/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 13s 3s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_acc
uracy: 1.0000 - val_loss: 0.0000e+00
Epoch 4/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 13s 4s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_acc
uracy: 1.0000 - val_loss: 0.0000e+00
Epoch 5/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 14s 4s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_acc
uracy: 1.0000 - val_loss: 0.0000e+00
Epoch 6/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 13s 3s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_acc
uracy: 1.0000 - val_loss: 0.0000e+00
Epoch 7/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 13s 3s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_acc
uracy: 1.0000 - val_loss: 0.0000e+00
Epoch 8/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 13s 3s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_acc
uracy: 1.0000 - val_loss: 0.0000e+00
Epoch 9/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 13s 3s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_acc
uracy: 1.0000 - val_loss: 0.0000e+00
Epoch 10/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 13s 3s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_acc
uracy: 1.0000 - val_loss: 0.0000e+00
Epoch 11/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 13s 3s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_acc
uracy: 1.0000 - val_loss: 0.0000e+00
Epoch 12/300
4/4 ━━━━━━━━━━━━━━━━━━━━ 13s 3s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_acc
uracy: 1.0000 - val_loss: 0.0000e+00
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step - accuracy: 1.0000 - loss: 0.0000e+00
Loss: 0.0
Accuracy: 1.0
```

Opcional: Guardamos el modelo en un archivo formato .h5 y lo cargamos en el dispositivo de despliegue

In [53]:
```
# Exportamos en modelo en un formato .h5
model.save('modelo_clasificacion_Alumno_Fondo.h5')
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.
saving.save_model(model)`. This file format is considered legacy. We recommend using
instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.savin
g.save_model(model, 'my_model.keras')`.
```

In [54]:
```
# Cargamos el modelo desde un archivo formato .h5
from keras.models import load_model
model = load_model('modelo_clasificacion_Alumno_Fondo.h5')
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be buil
t. `model.compile_metrics` will be empty until you train or evaluate the model.
```

2 Hacemos una predicción con un set de nuevas imagenes de la carpeta pruebas con fotos adicionales fuera de las usadas para entrenar y probar el modelo.

3 Foto de un rostro diferente al rostro del estudiante que no se encuentre en las carpetas de entrenamiento (el estudiante que presente antes de ti te dará un nombre de una celebridad que se parezca a ti, puedes descargarla de Google Images)

In [81]:

```
import cv2
import os
import numpy as np

# 1. Configuración de rutas y parámetros
ruta_pruebas = r'C:\MAESTRIA\VISION_COMPUTADORA_C1\CLASEII\Pruebas'
umbral = 0.5  # Punto de decisión para clasificar

# 2. Listar archivos de la carpeta
archivos = [f for f in os.listdir(ruta_pruebas) if f.lower().endswith(('.png', '.jp

print(f"--- Iniciando análisis de {len(archivos)} imágenes ---")

for nombre_archivo in archivos:
    # Construir ruta completa
    img_path = os.path.join(ruta_pruebas, nombre_archivo)

    # Lectura y Preprocesamiento
    img = cv2.imread(img_path)
    if img is None: continue

    # Redimensionar al tamaño que espera la VGG-16
    img_proc = cv2.resize(img, (224, 224))

    # IMPORTANTE: Convertir BGR a RGB si tu modelo fue entrenado con RGB
    img_proc = cv2.cvtColor(img_proc, cv2.COLOR_BGR2RGB)

    # Expandir dimensiones (Batch size de 1)
    img_proc = np.expand_dims(img_proc, axis=0)

    # 3. Realizar la predicción
    pred = model.predict(img_proc, verbose=0) # verbose=0 para no saturar la consol
    probabilidad = pred[0][0]

    # 4. Lógica de clasificación (Asumiendo 1=Alumno, 0=Fondo)
    if probabilidad >= umbral:
        resultado = "ALUMNO"
        certeza = probabilidad * 100
    else:
        resultado = "FONDO"
        certeza = (1 - probabilidad) * 100

    print(f"Imagen: {nombre_archivo} | Predicción: {resultado} ({certeza:.2f}% de c
```

```
print("--- Análisis finalizado ---")
```

```
--- Iniciando análisis de 5 imágenes ---
Imagen: Prueba13.jpg | Predicción: ALUMNO (100.00% de certeza)
Imagen: Prueba15.jpg | Predicción: FONDO (100.00% de certeza)
Imagen: Prueba19.jpg | Predicción: ALUMNO (100.00% de certeza)
Imagen: Prueba4.jpg | Predicción: ALUMNO (100.00% de certeza)
Imagen: Prueba9.jpg | Predicción: FONDO (100.00% de certeza)
--- Análisis finalizado ---
```

In [82]:

```
import cv2
import os
import numpy as np
import matplotlib.pyplot as plt

# 1. Configuración de la ruta
ruta_pruebas = r'C:\MAESTRIA\VISION_COMPUTADORA_C1\CLASEII\Pruebas'

# 2. Obtener la lista de imágenes en la carpeta
archivos = [f for f in os.listdir(ruta_pruebas) if f.lower().endswith(('.png', '.jp

print(f"Analizando {len(archivos)} imágenes...")

# 3. Bucle para procesar y visualizar cada imagen
for nombre_archivo in archivos:
    ruta_completa = os.path.join(ruta_pruebas, nombre_archivo)

    # Leemos la imagen
    img_bgr = cv2.imread(ruta_completa)
    if img_bgr is None:
        continue

    # Preprocesamiento para el modelo (Resize y Expand Dims)
    img_res = cv2.resize(img_bgr, (224, 224))
    img_batch = np.expand_dims(img_res, axis=0)

    # Realizamos la predicción
    pred = model.predict(img_batch, verbose=0)

    # Configuración de la visualización
    plt.figure(figsize=(6, 6))

    # Convertimos de BGR a RGB para Matplotlib
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    plt.imshow(img_rgb)

    # Lógica de etiquetado basada en las dos probabilidades de salida
    # pred[0][0] suele ser la clase 0 (Fondo) y pred[0][1] la clase 1 (Alumno)
    if pred[0][0] > pred[0][1]:
        titulo = f"Predicción: Fondo ({pred[0][0]*100:.1f}%)"
        color = 'red'
    else:
        titulo = f"Predicción: Alumno: Danny ({pred[0][1]*100:.1f}%)"
        color = 'green'
```

```
    plt.title(f"{nombre_archivo}\n{titulo}", color=color)
    plt.axis('off')
    plt.show()

print("Análisis completo.")
```

```
Analizando 5 imágenes...
```

Prueba13.jpg
Predicción: Alumno: Danny (100.0%)

Prueba15.jpg
Predicción: Alumno: Danny (100.0%)

Prueba19.jpg
Predicción: Fondo (100.0%)

Prueba4.jpg
Predicción: Fondo (100.0%)

Prueba9.jpg
Predicción: Alumno: Danny (100.0%)

```
Análisis completo.
```

Conclusiones Hemos realizado las pruebas a 20 imágenes de diferentes personas y al alumno con diferentes tipos de ropa, con lentes o sin lentes y el modelo a precedido de forma correcta el 100%. Esto ha sido posible también por que hemos entrenado al modelo con fotos del estudiante con diferentes ángulos, diferente ropa, diferentes accesorios, para que el modelo sea capaz de realizar la predicción de forma más acertada.